

图论

杨崇烨

上海交通大学

2026 年 7 月 5 日

目录

① 最短路

② 拓扑序

③ 生成树

Floyd 算法用于求任意两点间算短路, 适用于任何图 (不能有负环)。

- $f_{k,x,y}$, 只经过结点 1 到 k , x 到 y 的最短路。

Floyd 算法用于求任意两点间算短路, 适用于任何图 (不能有负环)。

- $f_{k,x,y}$, 只经过结点 1 到 k , x 到 y 的最短路。
- $f_{k,x,y} = \min(f_{k-1,x,y}, f_{k-1,x,k} + f_{k-1,k,y})$ 。

Floyd 算法用于求任意两点间算短路, 适用于任何图 (不能有负环)。

- $f_{k,x,y}$, 只经过结点 1 到 k , x 到 y 的最短路。
- $f_{k,x,y} = \min(f_{k-1,x,y}, f_{k-1,x,k} + f_{k-1,k,y})$ 。
- 省略第一维, $f_{x,y} = \min(f_{x,k} + f_{k,y}), O(n^3)$ 。

求非负权图上单源最短路径。

将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合，初始化 $dis(s) = 0$ ，其他均为 $+\infty$ 。

- 从 T 集合中，选取一个最短路长度最小的结点 x ，移到 S 集合中。

求非负权图上单源最短路径。

将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合，初始化 $dis(s) = 0$ ，其他均为 $+\infty$ 。

- 从 T 集合中，选取一个最短路长度最小的结点 x ，移到 S 集合中。
- 对 x 结点的所有出边执行松弛操作。

求非负权图上单源最短路径。

将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合，初始化 $dis(s) = 0$ ，其他均为 $+\infty$ 。

- 从 T 集合中，选取一个最短路长度最小的结点 x ，移到 S 集合中。
- 对 x 结点的所有出边执行松弛操作。
- 直到 T 集合为空，算法结束。时间复杂度 $O(n^2 + m)$ 。

求非负权图上单源最短路径。

将结点分成两个集合：已确定最短路长度的点集（记为 S 集合）的和未确定最短路长度的点集（记为 T 集合）。一开始所有的点都属于 T 集合，初始化 $dis(s) = 0$ ，其他均为 $+\infty$ 。

- 从 T 集合中，选取一个最短路长度最小的结点 x ，移到 S 集合中。
- 对 x 结点的所有出边执行松弛操作。
- 直到 T 集合为空，算法结束。时间复杂度 $O(n^2 + m)$ 。
- 使用 `priority_queue` 优化至 $O(m \log n)$ 。

求任意图单源最短路径, 可用于判负环。

初始化所有点 dis 为 inf , 源点设置为 0。进行 $n - 1$ 轮迭代, 对于每条边 $e = (u, v, w)$, 用 $dis_u + w$ 更新 dis_v 。

如果 n 轮之后仍有更新, 则有负环。时间复杂度 $O(nm)$ 。

SPFA 则是将 *Dijkstra* 的优先队列换成普通的队列。若当前要加入的点已在队列则不需要再次加入。

求任意图单源最短路径, 可用于判负环。

初始化所有点 dis 为 inf , 源点设置为 0。进行 $n - 1$ 轮迭代, 对于每条边 $e = (u, v, w)$, 用 $dis_u + w$ 更新 dis_v 。

如果 n 轮之后仍有更新, 则有负环。时间复杂度 $O(nm)$ 。

SPFA 则是将 *Dijkstra* 的优先队列换成普通的队列。若当前要加入的点已在队列则不需要再次加入。

~~SPFA 已死。~~

求有负权稀疏图上的全源最短路。Idea 是消除负权边, 跑 n 次堆优化 Dijkstra。复杂度 $O(nm \log m)$

求有负权稀疏图上的全源最短路。Idea 是消除负权边, 跑 n 次堆优化 Dijkstra。复杂度 $O(nm \log m)$

新建一个虚点 0, 向 $1 - n$ 所有点连边权为 0 的边。以 0 点跑一个 Bellman-Ford 求出 0 到每个点的距离 h_i 。对于 $e \in (u, v, w)$, 将其权值设置为 $w + h_u - h_v$ 。由于 $h_v \leq w + h_u$, 边权非负。则该图上 s 到 t 的最短路为 $f_{s,t} + h_s - h_t$ 。

Telephone G

n 个点, 每个点有属性 $k_i (1 \leq k_i \leq K)$, 给定了 $K \times K$ 的 01 矩阵 $s_{i,j}$ 。 i 号点能直接移动到 j 号点当且仅当 $s_{k_i, k_j} = 1$, 代价为 $|i-j|$ 。

求 1 号点到 n 号点的花费。

$n \leq 5 \times 10^4, K \leq 50$ 。

Snuke's Subway Trip

一个 n 个点 m 条边的无向图，每条边有一个属性 c_i ，一条路径的权值定义为边的序列中 c 的连续段数。求 1 到 n 的最短路径花费。

$n \leq 10^5, m \leq 2 \times 10^5$

Chase Game

给定一张 n 个点、 m 条边的无向无权连通简单图。Shou 初始在 1 号点，目标是到达 n 号点；Pang 初始在 k 号点，攻击范围为 d 。

Shou 每秒移动一步，同时 Pang 攻击 Shou。设此时两人在图上的距离为 dis ：

若 $dis < d$ ，Shou 受到 $d - dis$ 点伤害；

若 $dis \geq d$ ，Pang 传送到 Shou 当前所在点，并造成 d 点伤害。

求 Shou 从 1 号点到 n 号点过程中受到的最小总伤害。

$2 \leq n \leq 10^5$ ， $n - 1 \leq m \leq 2 \times 10^5$ ， $1 \leq k \leq n$ ， $1 \leq d \leq 2 \times 10^5$ 。

Elden Ring

给定一张包含 n 个点 m 条无向边的连通图。玩家初始在 1 号点，初始等级为 l_1 ，除 1 号点外每个点 i 有一个初始等级为 l_i 的 Boss。

每天开始时，所有未被击败的 Boss 等级增加 B 。随后玩家可以从 1 号点出发，沿着只经过已清空点的路径，到达某个仍有 Boss 的点并挑战它。若玩家当前等级严格大于该 Boss 等级，则击败该 Boss，玩家等级增加 A 。

玩家每天最多挑战一个 Boss。求最少多少天可以击败 n 号点的 Boss。

$2 \leq n \leq 2 \times 10^5$, $1 \leq m, A, B \leq 2 \times 10^5$

Meeting for Meals

给定一张 n 个点、 m 条带边权的无向连通图，1 号点是商场。共有 k 个朋友分别从互不相同的点 a_i 出发，所有人速度相同，且可以在边上的任意位置掉头。

设 t_{meet} 为所有朋友都能到达 1 号点的最早时间。在必须保证所有人都不晚于 t_{meet} 到达商场的前提下，对每个朋友 i 分别求：他最多能有多少时间与至少一个其他朋友待在同一位置。这里“同一位置”可以是某个点，也可以是边上的某个位置。

计算每个朋友答案时，其他朋友的路线都可以单独重新安排，只需让朋友 i 的陪伴时间最大。

$$2 \leq n \leq 3 \times 10^5, n-1 \leq m \leq 10^6, 2 \leq k \leq n$$

Exploration Boundary

给定一张 n 个点、 m 条边的无向连通简单图，源点固定为 1。定义 Dijkstra 运行过程中，已经被更新过、但最短路还未确定的点集，也就是当前优先队列 Q 中的点集，为一个探索边界。

给定 k 个期望出现的探索边界。你需要判断是否能给所有边赋权，使得：

- 每条边权都是 $[1, 10^9]$ 内的正整数。
- 从 1 出发到所有点的最短路长度两两不同。
- 这 k 个给定边界都能在同一次 Dijkstra 运行过程中出现。

若可以，输出 Yes，并按输入边的顺序输出一组合法边权；否则输出 No

$2 \leq n, m \leq 2 \times 10^5, 1 \leq k \leq 2 \times 10^5$

差分约束

差分约束系统是 n 元一次不等式组，包含 n 个变量 $x_1, x_2, \dots, x_n$ 以及 m 个约束条件。条件形如 $x_{a_i} - x_{b_i} \leq v_i$ ，求一组可行解。

看成是 $x_{a_i} \leq x_{b_i} + v_i$ ，这个限制等价于最短路中有一条 (b_i, a_i, v_i) 的边。从 0 向所有点连边权为 0 的边，跑单源最短路即可。有负环则无解。

Capitalism

n 个变量, m 条限制, 每条限制形如 $a_i = a_j + 1$ 或 $|a_i - a_j| = 1$ 。

判断是否有解, 并求一组极差最大的解。

$n \leq 200, n-1 \leq m \leq 2000$, 保证图连通

倍杀

n 名选手，每个人有个正实数分数，部分选手分数已知。若 A 的分数不小于 B 的 k 倍，称 A k 倍杀了 B 。

总共有 s 个 Flag:

- $o = 1$: A 立下“如果我没 k 倍杀 B ，我就吃顿好的”。
- $o = 2$: A 立下“如果 B 把我 k 倍杀，我就吃顿好的”

教练会选一个正常数 T 来放宽 Flag:

- $o = 1$ ，只要 A 成功 $(k - T)$ 倍杀 B ，就不用吃。
- $o = 2$ ，只要 A 没有被 B $(k + T)$ 倍杀，就不用吃。

在未知选手分数可以任意取正实数、但必须符合已知分数的前提下，求最大的实数 T ，使得无论最终分数如何，一定至少有一个选手会触发 Flag 去吃顿好的。

$n, s \leq 1000$

在一个 DAG(Directed Acyclic Graph, 有向无环图) 中, 我们将图中的顶点进行排序, 使得对于 $\forall e = (u, v)$, u 在 v 的前面。

一般通过 BFS 求解 DAG 的拓扑序。

deg_x 为 x 的入度, 用栈或队列存储入度为 0 的点, 每次从中取出一个点添加到拓扑序列尾部, 然后遍历其出边, 将连接到的点入度减一, 若减完为 0 将其加入栈或队列。

菜肴制作

n 个点 m 条限制，每条形如 (x, y) 表示 x 要排在 y 前面。要求在满足限制的情况下，1 尽可能靠前，然后 2 尽可能靠前，然后 3 . . . 。
 $n, m \leq 10^5$ 。

The Cow Gathering

n 个人形成树形的朋友关系，每个时刻会走一个人，要求若存在至少两个人则每个人都还有朋友未离开。对于每个人，求是否存在一个离开序列使得其最后离开。另有 m 条限制 (x, y) 表示 x 必须先于 y 离开。

$n, m \leq 10^5$ 。

Find the Circuit

通信题, 同一份程序会在每个测试中运行两次。

第一次运行时, 给一张 n 个点、 m 条边的连通无向简单图, 和简单环 $p_1, p_2, \dots, p_k$, 保证这些点按环上的顺序给定。你需要给图中每条边定向, 并输出得到的有向边。

第二次运行前, 评测机会随机重编号所有点, 并打乱第一次输出边的顺序。你需要按顺序给出新编号下的环, 起点任意。

输出环时允许循环移位, 但不能反向。

$3 \leq n \leq m \leq 5 \times 10^5$ $3 \leq k \leq n$ 。图连通, 无自环、无重边。

记忆，排列和有根树

给定一棵有根树和一个排列。对树上每个节点 u ，称其子树内一个节点 v 是重要的，当且仅当 $u \neq v$ ，并且在排列中， v 的出现位置在 u 的前面。令 d_u 为所有对 u 重要的节点和 u 的距离最小值，若没有则为 -1 。现在需要完成这道题的逆版本，给定有根树和 d_i 序列，求出字典序最小的排列使得通过上述方法可以算出对应的 d_i 序列，或者指出这样的排列不存在。数据范围： $n \leq 5 \times 10^5$ 。

Kruskal 算法求解最小生成树，复杂度为 $O(m \log m + m\alpha(n))$ ，瓶颈在排序。

算法流程为按边权从小到大遍历所有边，若该边的两头不在一个连通块就加入该边，用并查集维护图的连通性。正确性是若当前不加入该边连接这两头，这两头一定会在之后的某个时刻被连通，那不如用这条边替换之后的那条边。

Prim 算法求解最小生成树，朴素实现复杂度为 $O(n^2 + m)$ ，使用数据结构优化寻找最小值复杂度为 $O(n \log n + m \log n)$ 。

选择一个点为初始点距离为 0，其他点的距离设置为 ∞ 。每次找出最近点拓展，更新与其连边点的距离。

证明考虑归纳命题：在每一步，都有一颗最小生成树包含已选边集合。

Boruvka 算法求解最小生成树，复杂度为 $O(m \log n)$ 。

进行至多 $O(\log n)$ 轮迭代，对于每个连通块，找到与其距离最近的另一个连通块，加入连接边。

最坏情况连通块个数也会减半。其实就是并行的 Prim。

造桥与砍树

n 个点, 每个点有点权 t_i , i 和 j 之间的边权为 $(t_i + t_j) \% k$ 。求最小生成树
 $1 \leq n \leq 10^5, 1 \leq t_i, k \leq 10^9$

最小公倍树

有一个节点编号为 $[L, R]$ 的完全图, (i, j) 边的边权为 $\text{lcm}(i, j)$, 求最小生成树。
 $1 \leq L \leq R \leq 10^6, R-L \leq 10^5$

新年的繁荣

给定 n, m 和长度为 n 的序列 $a_i (0 \leq a_i < 2^m)$ 。 (i, j) 连边的代价为 a_i and a_j ，即按位与。求最小生成树。

$n \leq 10^5, m \leq 18$ 。

Optimization for UltraNet

n 个点、 m 条边的无向连通图，每条边有互不相同的带宽 b 。现在要按以下优先级优化网络：

- 保证图连通且保留边数最少。
- 最大化整棵树的瓶颈值。树上任意两点间带宽定义为路径上最小边权，整棵树瓶颈值即所有点对带宽中的最小值。
- 在满足前两条的前提下，最小化保留边的带宽总和。

求最终优化出的树中，所有城市点对 (i, j) 的带宽之和。也就是对每一对点，取它们在树上路径中的最小边权，累加所有点对的结果

$$n \leq 10^4, m \leq 5 * 10^5$$

即将 Kruskal 的过程建成树，当加入边 (u, v, w) 的时候，在新图上新建节点 t ，点权为 w ，然后连接 $(t, \text{get}(u))$ 和 $(t, \text{get}(v))$ ，再设置 $\text{fa}[\text{get}(u)] = \text{fa}[\text{get}(v)] = t$ 。

这样建出来的树， u, v 的最近公共祖先的点权即为连通 u, v 的所使用的边中边权最大值的最小值。

归程

给定一张 n 个点、 m 条边的无向连通图，1 号点是家。每条边有两个属性：长度 l 和海拔 a 。当某天水位线为 p 时，所有海拔 $a \leq p$ 的边都会积水。

Q 天，每天给出出发点 v 和水位线 p 。Yazid 从 v 开车出发，车只能经过没有积水的边。Yazid 可以在任意点下车，之后改为步行回到 1 号点，步行可以经过任意边，代价为步行经过边的长度和。每天的车都会重置，不能利用前一天停下的车。

对每一天，求 Yazid 回家所需的最小步行总长度。

强制在线。 $n \leq 2 \times 10^5, m, Q \leq 4 \times 10^5$

Life is a Game

给定一张 n 个点、 m 条边的无向连通图。每个点 i 有收益 a_i ，每条边有能力要求 w 。

有 q 次询问，每次给定初始点 x 和初始能力值 k 。玩家一开始没有在任何城市获得过收益；到达一个城市后，可以获得该城市的 a_i 点能力，但每个城市最多只能获得一次。

玩家可以经过一条边，当且仅当当前能力值不小于该边的能力要求 w ，通过边不会消耗能力。

对每次询问，求玩家最终最多能获得多少能力值。

$n, m, q \leq 10^5$

Mountain Booking

给定一棵 n 个点的带权树，表示第 0 天的山路地图。接下来 m 天中，每天早上会永久关闭一条已有道路，并新建一条带权道路，且修改后地图始终仍是一棵树。

共有 p 个游客预订：第 i 个游客会在第 a_i 天下午前往区域 b_i 。定义当前树上两点 u, v 之间的： $f(u, v) = u$ 到 v 唯一路径上的最大边权。

有 q 次询问，每次给定 c, d ，要求计算第 c 天中，所有预订去其他区域的游客的 $\sum f(b_j, d)$
 $n \leq 10^5, m, p, q \leq 2 \times 10^5$

Cheap Robot

n 个点 m 条边的无向带权连通图，前 k 个点可将电充满。每经过一条边权为 w 的边会消耗 w 电量， q 次询问从 a 走到 b 电池容量至少为多少？保证 $a, b \leq k$ 。

$1 \leq k \leq n \leq 10^5, m, q, \leq 3 \times 10^5, w_i \leq 10^9$

Travel 2

交互题。隐藏着一张 n 个点、 m 条边的无向连通简单图，小青鱼从 1 号城市出发，但一开始完全不知道有哪些边，目标是找出图中的所有道路。

每次交互时，评测机会告诉你当前所在城市编号 x 和它的度数 d_x 。你可以选择走当前城市的第 i 条路， $1 \leq i \leq d_x$ 。每个城市的道路顺序未知但固定。走完后，会到达这条边的另一端，并得到新城市编号和度数。

总行走次数不能超过 $2m + 2n$ 。最后输出所有找到的 m 条边，边的顺序任意，输出答案本身不计入次数限制。

$n \leq 2500, m \leq 10000$

Information Spread

给定一张 n 个点、 m 条边的有向图，点表示学生。最开始只有 1 号学生知道信息。每条有向边 $u \rightarrow v$ 有传播概率 $w = p/q$ 。

信息传播过程按一次 DFS 进行：从 1 号点开始，点只会被 DFS 访问一次；遍历某个点 u 的出边时，必须按照输入顺序。对于边 $u \rightarrow v$ ，若此时 u 已知道信息且 v 还不知道，则有 w 的概率让 v 知道信息；之后无论是否传播成功，都会继续 DFS 到 v 。

注意：如果一个点在“不知道信息”时已经被 DFS 访问过，之后即使它又知道了信息，也不会再次从它出发传播。

求整个过程结束后，每个学生知道信息的概率，并对 998244353 取模输出。

$n \leq 10^5, n-1 \leq m \leq 3 \times 10^5, 0 \leq p \leq q \leq 10^5$